kontaktstudium informatik koin

Themenbereich C
Lerneinheit C2
Übungsaufgaben

Hallo und herzlich willkommen zum 2. Übungsblatt zum Thema Rechnerarchitektur!

In diesem Übungsblatt wollen wir die Hardware-Komponenten der Logik in den Mittelpunkt rücken. Dazu werden wir die Konzepte nutzen, die wir im letzten Übungsblatt gelernt haben.
Der Schwerpunkt der ersten Aufgaben liegt auf der Einführung des Halb- und Vollsubtrahierers bzw. -addierers. Die Subtraktion lässt sich dabei auch als Addition von negativen Zahlen darstellen. In der Realität werden die negativen Äquivalente der Zahlen, wie gelernt, gebildet und mit diesen addiert. Auf diese Weise kommen große Schaltanlagen mit einem einzigen Rechenwerk aus, das wahlweise addieren oder subtrahieren kann. Das führt zu erheblichen Einsparungen.

Viel Spaß beim Ausprobieren und Knobeln!

Transferaufgabe

Programmieraufgabe

Anwendungsaufgabe

Mögliche Klausuraufgabe

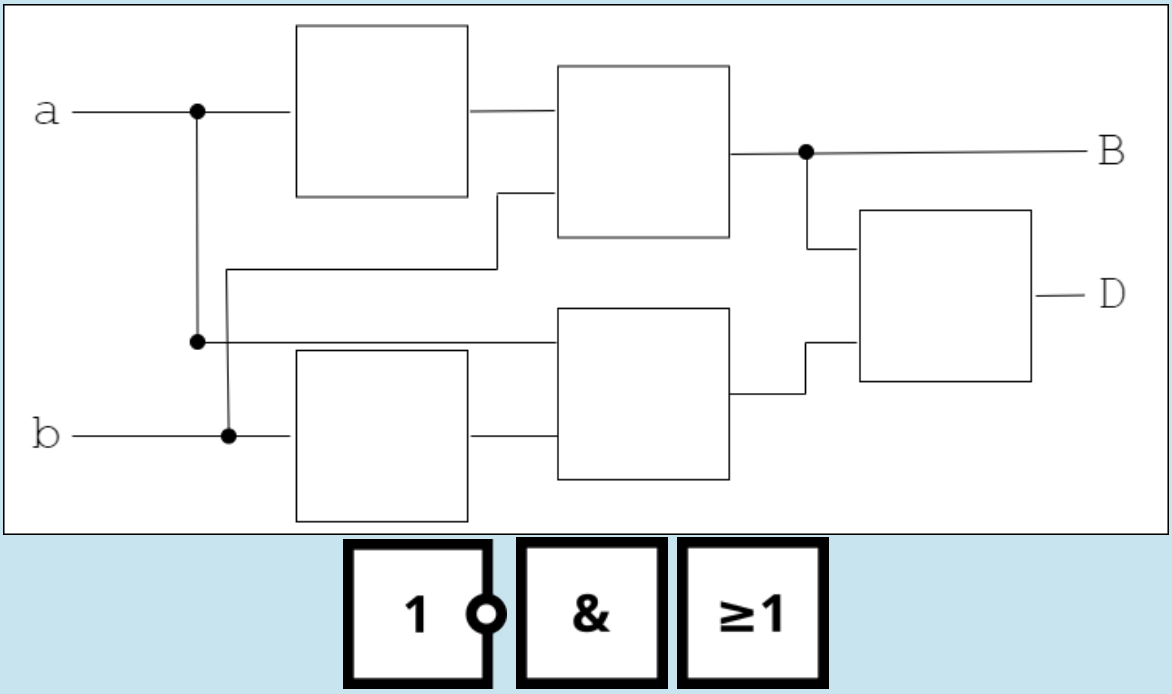
Aufgabe 1: Halbsubtrahierer (3 Punkte)

In dieser Aufgabe werden wir einen Halbsubtrahierer genauer in Augenschein nehmen. Wie bei der binären Addition entsteht bei der Subtraktion a – b zweier Binärziffern a, b gegebenenfallsein negativer Übertrag, den wir hier **Borger** nennen wollen (ein von der nächsthöheren Bitposition „geborgtes“ Bit, engl. „borrow bit“).

Ergänzen Sie die Wahrheitstabelle des Halbsubtrahierers um die Werte für die Differenz **D** und den Borger **B** für a - b:

a	b	D	B
0	0		
0	1		
1	0		
1	1		

Ergänzen Sie die untenstehende Gatterschaltung für den Halbsubtrahierer.



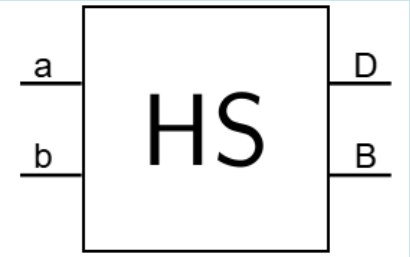
Aufgabe 2: Vollsubtrahierer (4 Punkte)

In dieser Aufgabe geht es um einen Vollsubtrahierer für zwei zu subtrahierende Binärzahlen

$x - y$. Als weiteren Eingang hat man außerdem einen eventuell eingehenden Borger B_{in} aus einer vorigen Rechnung. Die zugehörige Wahrheitstabelle für die Differenz D und den ausgehenden Borger B_{out} ist dabei wie folgt gegeben:

x	y	B_{in}	D	B_{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Für die Darstellung des Vollsubtrahierers dürfen Sie das nachfolgende Gatter als Halbsubtrahierer verwenden:



Geben Sie eine Gatterschaltung für einen Vollsubtrahierer an, unter Verwendung des obigen Halbsubtrahierers. Beschriften Sie die Ausgänge jedes Halbsubtrahierer mit B und D, sodass klar ist, bei welchem Ausgang es sich um den Borger bzw. die Differenz handelt.

Akzeptierte Dateitypen:
Bilddatei (JPEG) .jpeg .jpg, Bilddatei (PNG) .png, Bilddatei (SVG+XML) .svg, PDF-Dokument .pdf,

Aufgabe 3: Addierwerk und Volladdierer (3 Punkte)

Ein Addierwerk ermöglicht die Addition zweier (mehrstelliger) Binärzahlen. Da sich alle vier Grundrechenarten auf die Addition zurückführen lassen, bildet es das zentrale Element des Rechenwerks (Arithmetisch-logische Einheit, ALU) eines Prozessors. In dieser Aufgabe möchten wir ein Addierwerk für dreistellige Binärzahlen analysieren und anschließend als Gatterschaltung aufbauen.

Zum Erstellen der Gatterschaltung nutzen wir den in der Vorlesung eingeführten Volladdierer. Dieser ermöglicht es zwei Binärziffern (=Bits) (a_i, b_i) und einen Übertrag (c_{in}) zu addieren. Als Ergebnis wird die Summe s und ein Übertrag (c_{out}) ausgegeben (siehe vollständige Tabelle in Teilaufgabe b). Da ein Volladdierer nur je zwei Ziffern und ggfs. einen Übertrag addieren kann, brauchen wir für dreistellige Binärzahlen auch mehrere Volladdierer. Jeder Volladdierer führt dann die Addition einer Stelle aus und gibt die Ergebnisse an die nächste Stelle und damit an den nächsten Volladdierer weiter. Um die Gatterschaltung zu analysieren und aufzubauen, schauen wir uns im ersten Schritt die konkrete Addition zweier Binärzahlen an. Die drei Farben grün, rot und gelb geben dabei jeweils eine Stelle bzw. eine Binärziffer an, die später durch einen Volladdierer repräsentiert wird:

$$a_2a_1a_0 + b_2b_1b_0 = 1\,0\,1_2 + 1\,1\,1_2$$

Eine schriftliche Addition haben Sie bereits in früheren Lerneinheiten schon durchgeführt. Wie man in folgendem Beispiel sieht, werden 4 Bits benötigt um das Ergebnis der Addition richtig darzustellen, da der letzte Übertrag berücksichtigt werden muss. Bei der Umsetzung kommen wir später trotzdem mit drei Volladdierern aus, da der entsprechende Übertrag durch den letzten Volladdierer direkt mit ausgegeben wird.

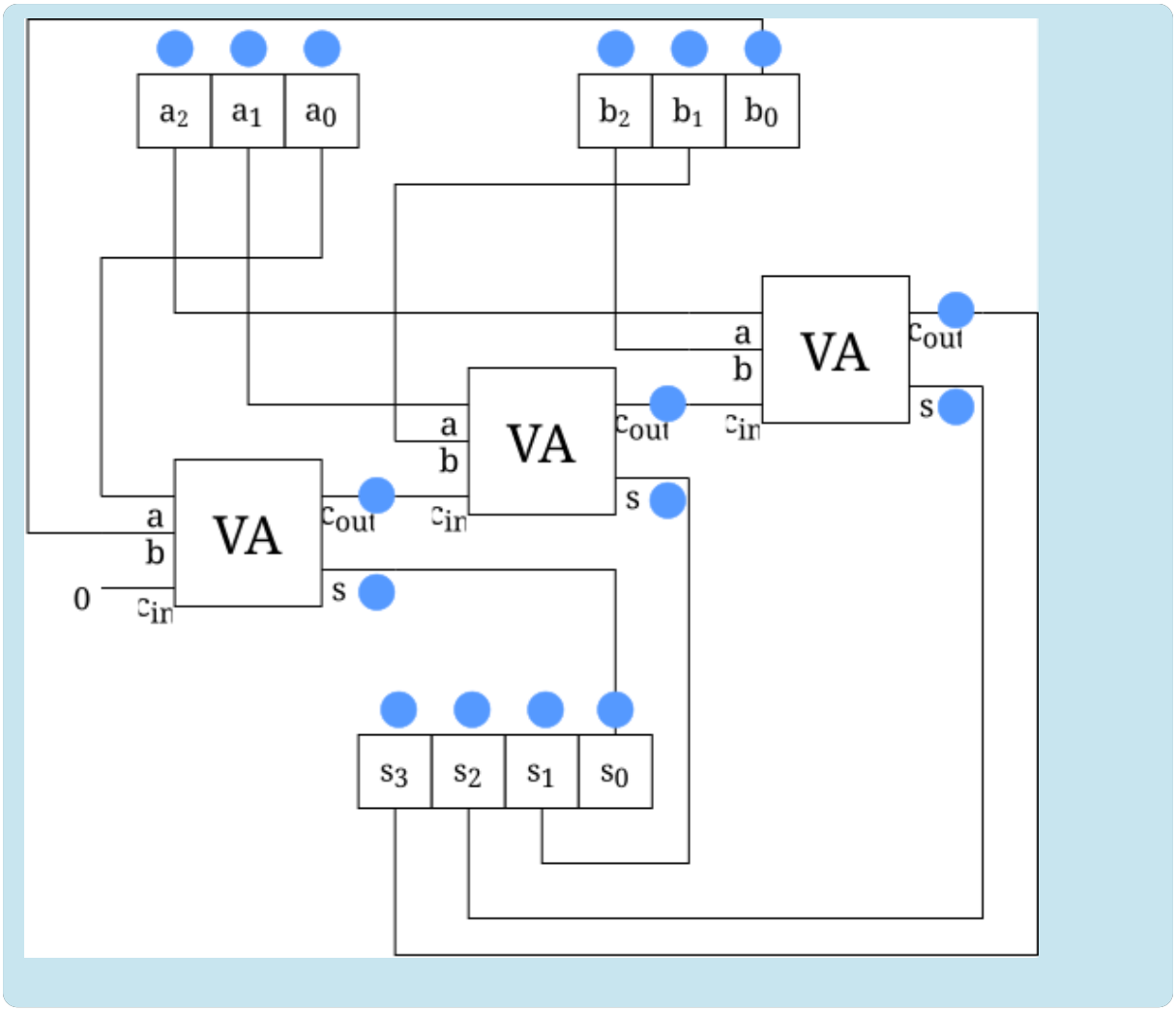
$$\begin{array}{lccc} & 1 & 0 & 1 \\ +_1 & 1_1 & 1_1 & 1 \\ \hline 1 & 1 & 0 & 0 \end{array}$$

Schauen wir uns nun die Wahrheitstabelle des Volladdierers genauer an. Für jede Kombination der Eingänge (ai , bi und c_{in}) wird dabei die berechnete Summe s und der Übertrag c_{out} dokumentiert.

Ihre Aufgabe: Markieren Sie in der gegebenen Tabelle in der letzten Spalte durch die Wörter grün, rot bzw. gelb , oder nichts (-), welche Zeilen der Tabelle den Stellenwerten bei der Rechnung oben entsprechen.

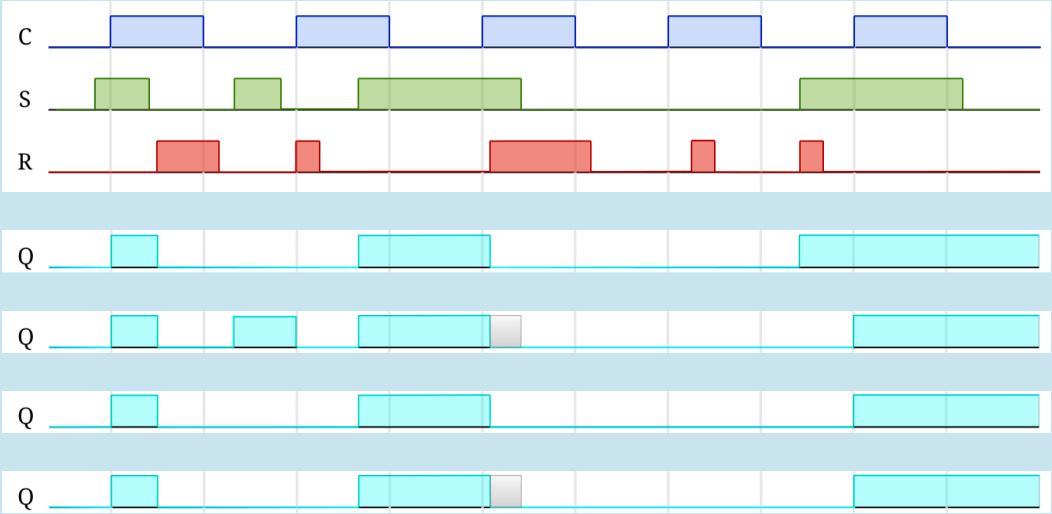
a_i	b_i	c_{in}	s	c_{out}	
0	0	0	0	0	Select: ▼ grün | rot | gelb | -
0	0	1	1	0	Select: ▼ grün | rot | gelb | -
0	1	0	1	0	Select: ▼ grün | rot | gelb | -
0	1	1	0	1	Select: ▼ grün | rot | gelb | -
1	0	0	1	0	Select: ▼ grün | rot | gelb | -
1	0	1	0	1	Select: ▼ grün | rot | gelb | -
1	1	0	0	1	Select: ▼ grün | rot | gelb | -
1	1	1	1	1	Select: ▼ grün | rot | gelb | -

Jetzt wird es spannend(er): unten ist ein Addierwerk abgebildet, um die o.g. Berechnung durchzuführen. Die Gatterschaltung ist mit Hilfe von Volladdierern umgesetzt. Damit kann sie nicht nur die konkrete Berechnung von $101_2 + 111_2$ durchführen, sondern zwei beliebige dreistellige Binärziffern addieren. Durch den Übertrag des letzten Volladdierers ist das Ergebnis eine vierstellige Binärzahl. Ganz konkret können mit der Gatterschaltung also die Additionen von $000_2 + 000_2$ bis $111_2 + 111_2$ richtig durchgeführt werden:
$000_2 + 000_2 = 0000_2$
$111_2 + 111_2 = 1110_2$
Allgemein also: $a_2a_1a_0 + b_2b_1b_0 = s_3s_2s_1s_0$
In der Schaltung sind die sechs Eingänge $a_2, a_1, a_0, b_2, b_1, b_0$ sowie die vier Ausgänge s_3, s_2, s_1, s_0 markiert, sowie für jeden Volladdierer, bei welchem Ausgang es sich um die Summe S und den Übertrag C_{out} handelt.
Ihre Aufgabe: Dokumentieren Sie die Rechnung $101_2 + 111_2$ in der Gatterschaltung, indem Sie analog zur Vorlesung (Minute 5:20) (Gatterschaltung mit konkreten Eingangssignalen) jeden Eingang und jeden Ausgang jedes Gatters mit dem entsprechenden Wahrheitswert (0 oder 1) annotieren. Ziehen Sie dazu den jeweils passenden Marker in die dafür vorgesehenen blauen Felder (Funktion nur in Moodle verfügbar).

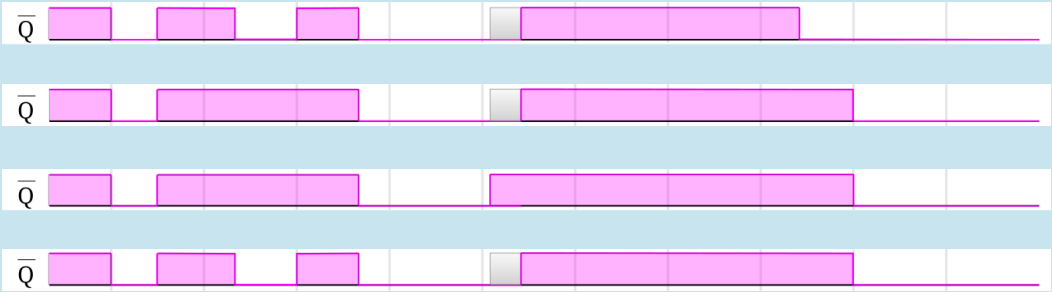

a2
a1
a0
b2
b1
b0
VA
a
b
cout
cin
s
VA
a
b
cout
cin
s
VA
a
b
0
cin
cout
s
s3
s2
s1
s0

Aufgabe 4: SR-Riegel und Takt (5 Punkte)

Wählen Sie den korrekten Verlauf für Q, passend zur Vorgabe bei C, S und R:

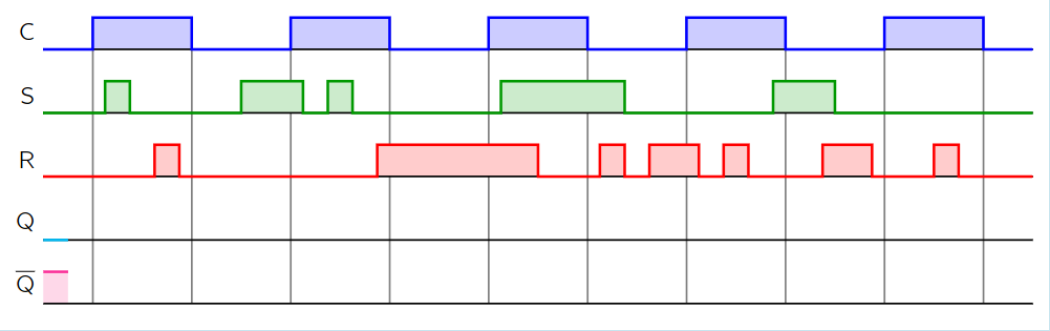


Wählen Sie den korrekten Verlauf für $\bar{Q}$, passend zur Vorgabe bei C, S und R aus der vorigen Teilaufgabe.



Ergänzen Sie „von Hand“ im untenstehenden Schaubild Q und $\bar{Q}$.

Hinweis: Achten Sie darauf, dass das Zusammenspiel der verschiedenen Zustände und deren zeitlicher Verlauf eindeutig erkennbar sind.



Akzeptierte Dateitypen:
Bilddatei (JPEG) .jpeg .jpg, Bilddatei (PNG) .png, Bilddatei (SVG+XML) .svg, PDF-Dokument .pdf,

Themenbereich D

Bitte beachten Sie: Die folgende Aufgabe mit insgesamt 5 Punkten zählt zum Themenbereich D: Praktische Implementation von Algorithmen (vgl. Merkblatt zur Prüfungszulassung).

Aufgabe 5: Budgetplanung in Java (5 Punkte)

Für ein Upgrade ihres Büros möchte eine motivierte Informatik-Lehrkraft einige neue Einrichtungs- und Dekorationsgegenstände aus einem schwedischen Möbelhaus erwerben. Dafür sucht sie online nach interessanten Produkten und notiert sich deren Preise in der Reihenfolge, in der ihr die Angebote über den Weg laufen in einem Array.

Da ihr Ziel ist, möglichst viele neue Gegenstände für ein Klassenzimmer zu erwerben, das Budget aber selbstverständlich beschränkt ist, schreibt die Lehrkraft ein Java-Programm, das ihr bei der Budgetplanung helfen soll.

Das folgende Programm zeigt den Stand nach der Sichtung der Angebote. Im Array ***preise*** sind in willkürlicher Reihenfolge die Preise der interessanten Produkte eingetragen worden. Um sich einen besseren Überblick über die Kosten und das Budget zu verschaffen, soll das Array nicht-absteigend, also vom günstigsten bis zum teuersten Preis, sortiert werden.

```
int[] preise = {30, 15, 105, 450, 20, 180, 50, 320, 20};
System.out.println("Unsortiert:");
System.out.println(Arrays.toString(preise));

//die Methode "sort" arbeitet direkt auf dem Array, da es sich um einen Referenzdatentyp handelt
//somit ist kein Rückgabewert nötig und die Methode kann im folgenden einfach aufgerufen werden:
System.out.println("Sortieren:");
sort(preise);
```

Zu diesem Zweck wurde bereits eine Methode ***sort*** implementiert, die ein int-Array als Eingabe erhält und dieses entsprechend sortieren soll. Der Code ist unten im Eingabefenster gegeben. Sie können den gesamten Code außerdem in Moodle herunterladen, um ihn in VSC zu testen.

Leider sind beim Programmieren ein paar Dinge schief gelaufen, wodurch das Programm noch nicht erfolgreich ausgeführt werden kann...

Ihre Aufgabe: Finden Sie die **vier Syntaxfehler**, die sich in die Methode eingeschlichen haben. Beheben Sie außerdem **den semantischen Fehler**, der gemacht wurde und zu einem unerwünschten Ergebnis führt.

Sie können die Fehler direkt im Code-Fenster beheben und mithilfe des Prüfen-Buttons testen, ob die Methode funktioniert. Der Abschnitt „Zum Beispiel“ zeigt wie gewohnt beispielhaft, welche Ergebnisse erwartet werden. Ändern Sie abgesehen von der Fehlerbehebung daher nichts am Programm ab.

Bei Bedarf können Sie per Kommentar im Codefenster weitere Erklärungen zu Ihren gefundenen Fehlern notieren. Dies ist aber für die volle Punktzahl nicht notwendig.

Zum Beispiel:

Test	Resultat

`int[] preise = {30, 15, 105, 450, 20, 180, 50, 320, 20};`
`sort(preise);`
	`[30, 15, 105, 20, 20, 180, 50, 320, 450]`
	`[30, 15, 105, 20, 20, 180, 50, 320, 450]`
	`[30, 15, 105, 20, 20, 50, 180, 320, 450]`
	`[30, 15, 50, 20, 20, 105, 180, 320, 450]`
	`[30, 15, 20, 20, 50, 105, 180, 320, 450]`
	`[20, 15, 20, 30, 50, 105, 180, 320, 450]`
	`[20, 15, 20, 30, 50, 105, 180, 320, 450]`
	`[15, 20, 20, 30, 50, 105, 180, 320, 450]`
	`Sortiert:`
	`[15, 20, 20, 30, 50, 105, 180, 320, 450]`

`int[] preise = {1, 2, 3, 4, 5};`
`sort(preise);`
	`[1, 2, 3, 4, 5]`
	`[1, 2, 3, 4, 5]`
	`[1, 2, 3, 4, 5]`
	`[1, 2, 3, 4, 5]`
	`Sortiert:`
	`[1, 2, 3, 4, 5]`

Antwort: (Abzugssystem: 0 %)

```
void sort(int[] list) {
    int n = list.length-1;
    int memory = 0;

    while(n > 0) {
        int max_index = 0;
        for(i = 0; i <= n ; i++) {
            if(list[i] <= list[max_index]){
                max_index = i;
            }
        }

        memory = list(n);
        list[n] = list[max_index];
        list[max_index] = memory;

        n = n - 1

        System.out.println(Arrays.toString(list));
    }
    System.out.println("Sortiert:");
    System.out.println(Arrays.toString(list));
```

Nachdem die Preise nun mithilfe der reparierten ***sort***-Methode sortiert angezeigt werden können, stellt sich die Frage, wie viele Produkte mit einem bestimmten Budget eingekauft werden können. Zu diesem Zweck soll eine weitere Methode angelegt werden, die **die Elemente des Arrays *preise* schrittweise aufaddiert**. Aus einem Array {1, 2, 3, 4, 5} soll also als Ergebnis dieser Methode {1, 3, 6, 10, 15} gemacht werden. So steht an Stelle ***i*** (gezählt ab Index 0) des Arrays der Gesamtpreis für die günstigsten i+1 Produkte.

Ihre Aufgabe: Schreiben Sie eine Methode namens ***addElements*** ohne Rückgabewert, die analog zur Methode ***sort*** ein int-Array als Eingabe erhält und direkt auf dieser Eingabe arbeitet. Die Methode soll wie oben beschrieben die Elemente des Arrays schrittweise aufaddieren. Dafür gibt es unterschiedliche Möglichkeiten - welchen algorithmischen Weg Sie konkret wählen, bleibt Ihnen überlassen. Wichtig ist, dass die Methode für alle beliebigen Testfälle funktioniert und für jeden ein wie im Abschnitt „Zum Beispiel" gezeigtes Ergebnis erzeugt.

Zum Beispiel:

Test	Resultat
`int[] preise = {1, 2, 3, 4, 5};` `addElements(preise);`	`[1, 3, 6, 10, 15]`
`int[] preise = {15, 20, 20, 30, 50, 105, 180, 320, 450};` `addElements(preise);`	`[15, 35, 55, 85, 135, 240, 420, 740, 1190]`

Antwort: (Abzugssystem: 0 %)

```
//Hier können Sie die addElements-Methode abgeben:
```